Matthew Tran
USB Drive Partitioning & Memory Layout Analysis

In this lab, I'll do two things: first, I'll prepare a USB drive for use in Linux by partitioning, formatting, and mounting it, then examine its file system details. Second, I'll look at how a C program uses memory, stepping through code with a debugger to see how functions and variables are stored on the stack.

Tools Used

- Linux terminal
- dmesg
- fdisk
- mkfs (ext4 and vfat)
- dumpe2fs
- mount / umount
- gcc compiler
- size command
- objdump
- GDB debugger
- gedit text editor

Skills Learned

- Partitioning a USB drive using fdisk
- Formatting partitions with ext4 and vfat file systems
- Mounting and unmounting storage devices in Linux
- Analyzing file system details using dumpe2fs
- Compiling C programs and examining their memory layout with size and objdump
- Using GDB to set breakpoints, step through code, and inspect the call stack
- Understanding stack frames and function-level memory management
- Debugging C programs to observe variable behavior and execution flow

Task 1) Partitioning, Formatting, and Mounting USB Stick on Linux

In this part of the lab, I learned how to prepare a USB stick for use in a Linux environment by partitioning, formatting, and mounting it. This process is essential for managing storage devices in Linux and understanding how file systems work.

1. **Check if the device was mounted**

```
matthew_tran@vm:~$ dmesg | tail
[  204.600912] usbcore: registered new interface driver uas
[  205.619908] scsi 3:0:0:0: Direct-Access     General  UDisk            5.00 PQ: 0 ANSI: 2
[  205.620337] sd 3:0:0:0: Attached scsi generic sg2 type 0
[  205.637948] sd 3:0:0:0: [sdb] 15730688 512-byte logical blocks: (8.05 GB/7.50 GiB)
[  205.650101] sd 3:0:0:0: [sdb] Write Protect is off
[  205.650103] sd 3:0:0:0: [sdb] Mode Sense: 0b 00 00 08
[  205.661850] sd 3:0:0:0: [sdb] No Caching mode page found
[  205.661853] sd 3:0:0:0: [sdb] Assuming drive cache: write through
[  205.767963]  sdb: sdb1
[  205.841367] sd 3:0:0:0: [sdb] Attached SCSI removable disk
matthew_tran@vm:~$
```

dmesg | tail

The dmesg | tail command shows the last few lines of the system's logs, including information about newly attached devices. This step helps identify the device name of the USB stick.

2. **If the USB is mounted automatically, unmount it:**

```
matthew_tran@vm:~$ sudo umount /dev/sdb1
matthew_tran@vm:~$
```

sudo umount /dev/sdb1

If the device is automatically mounted, we'll need to unmount it using unmount to prevent data corruption during partitioning. This ensures that no processes are accessing the USB drive while you make changes to its partitions.

3. **Partition the USB stick:**

sudo fdisk /dev/sdb1

```
matthew_tran@vm:~$ sudo fdisk /dev/sdb1

Welcome to fdisk (util-linux 2.34).
Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.


Command (m for help): 
```

```
Welcome to fdisk (util-linux 2.34).
Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.


Command (m for help): p
Disk /dev/sdb1: 7.51 GiB, 8053063680 bytes, 15728640 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x500a0dff

Device      Boot      Start        End    Sectors   Size Id Type
/dev/sdb1p1      1948285285 3650263507 1701978223 811.6G 6e unknown
/dev/sdb1p2             0          0          0     0B 74 unknown
/dev/sdb1p4        28049408   28049848        441 220.5K  0 Empty

Partition table entries are not in disk order.

Command (m for help): 
```

```
Command (m for help): p
Disk /dev/sdb1: 7.51 GiB, 8053063680 bytes, 15728640 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x500a0dff

Device      Boot      Start        End    Sectors   Size Id Type
/dev/sdb1p1      1948285285 3650263507 1701978223 811.6G 6e unknown
/dev/sdb1p2               0          0          0     0B 74 unknown
/dev/sdb1p4        28049408   28049848        441 220.5K  0 Empty

Partition table entries are not in disk order.

Command (m for help): d
Partition number (1,2,4, default 4): 1

Partition 1 has been deleted.

Command (m for help): d
Partition number (2,4, default 4): 2
Could not delete partition 2

Command (m for help): d
Partition number (2,4, default 4): 4

Partition 4 has been deleted.

Command (m for help): d
Selected partition 2
Could not delete partition 2

Command (m for help): n
Partition type
   p   primary (0 primary, 0 extended, 4 free)
   e   extended (container for logical partitions)
Select (default p): p
Partition number (1,3,4, default 1): 1
First sector (2048-15728639, default 2048):
Last sector, +/-sectors or +/-size{K,M,G,T,P} (2048-15728639, default 15728639):

Created a new partition 1 of type 'Linux' and of size 7.5 GiB.

Command (m for help):
```

```
Partition type
   p   primary (0 primary, 0 extended, 4 free)
   e   extended (container for logical partitions)
Select (default p): p
Partition number (1,3,4, default 1): 1
First sector (2048-15728639, default 2048):
Last sector, +/-sectors or +/-size{K,M,G,T,P} (2048-15728639, default 15728639):

Created a new partition 1 of type 'Linux' and of size 7.5 GiB.

Command (m for help): w
The partition table has been altered.
Failed to remove partition 1 from system: Invalid argument
Failed to remove partition 4 from system: Invalid argument
Failed to add partition 1 to system: Invalid argument

The kernel still uses the old partitions. The new table will be used at the next reboot.
Syncing disks.

matthew_tran@vm:~$ w
 19:19:42 up 8 min,  1 user,  load average: 0.12, 0.16, 0.09
USER     TTY      FROM             LOGIN@   IDLE   JCPU   PCPU WHAT
seed     :0       :0               19:11   ?xdm?  16.30s  0.00s /usr/lib/gdm3/gdm-x-session --run-script env GNOME_SHELL_SESSION_MODE=ubuntu /us
matthew_tran@vm:~$ sudo fdisk /dev/sdb1

Welcome to fdisk (util-linux 2.34).
Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.


Command (m for help): p
Disk /dev/sdb1: 7.51 GiB, 8053063680 bytes, 15728640 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x500a0dff

Device      Boot Start      End  Sectors  Size Id Type
/dev/sdb1p1       2048 15728639 15726592  7.5G 83 Linux
/dev/sdb1p2          0        0        0    0B 74 unknown

Partition table entries are not in disk order.

Command (m for help): w
The partition table has been altered.
Syncing disks.

matthew_tran@vm:~$
```

Inside the fdisk tool:

Type p to display current partitions.

Type d to delete the existing partition.

Type n to create a new partition.

Type p to select a primary partition.

Type 1 to assign it as the first partition.

Press Enter to accept the default starting and ending locations.

Type w to save the partition table and exit fdisk.

The fdisk command is used to create or modify partitions on the USB stick. Partitioning prepares the USB stick for formatting by defining a specific area on the device where a file system can stay/be found.

4. **Format the new partition:**

```
matthew_tran@vm:~$ sudo mkfs -t ext4 /dev/sdb1
mke2fs 1.45.5 (07-Jan-2020)
/dev/sdb1 contains a vfat file system labelled 'ESD-USB'
Proceed anyway? (y,N) y
Creating filesystem with 1966080 4k blocks and 491520 inodes
Filesystem UUID: 3eda08ca-a403-474e-adf6-bd431206a61a
Superblock backups stored on blocks:
        32768, 98304, 163840, 229376, 294912, 819200, 884736, 1605632

Allocating group tables: done
Writing inode tables: done
```

sudo mkfs -t ext4 /dev/sdb1

The mkfs -t ext4 command creates a new ext4 file system on the partition, which is the format used by Linux. This step is necessary to prepare the USB stick for use with Linux by orgainizing its storage according to the ext4 format.

5. **Create a mount point:**

```
matthew_tran@vm:~$ mkdir mediatest1
```

mkdir mediatest1

The mkdir command creates a directory (mount point) where the USB stick's file system can be attached to the Linux directory structure. This is necessary because Linux requires a mount point to access files on an external device.

6. **Mount the new file system:**

```
matthew_tran@vm:~$ sudo mount -t ext4 /dev/sdb1 mediatest1
```

sudo mount -t ext4 /dev/sdb1 mediatest1

Check if the mount was successful:

```
matthew_tran@vm:~$ ls mediatest1
lost+found
```

ls mediatest1

The mount command attaches the USB stick's file system to the mount point we've created. This allows us to interact with the USB stick's files and directories.

7. **Check file system details:**

```
[09/27/24]seed@VM:~$ sudo dumpe2fs -h /dev/sdb1
dumpe2fs 1.45.5 (07-Jan-2020)
Filesystem volume name:   <none>
Last mounted on:          <not available>
Filesystem UUID:          91a18c2f-b113-4e1f-8d1b-2c4ddf095237
Filesystem magic number:  0xEF53
Filesystem revision #:    1 (dynamic)
Filesystem features:      has_journal ext_attr resize_inode dir_index filetype needs_recovery extent 64bit flex_bg sparse_su
per large_file huge_file dir_nlink extra_isize metadata_csum
Filesystem flags:         signed_directory_hash
Default mount options:    user_xattr acl
Filesystem state:         clean
Errors behavior:          Continue
Filesystem OS type:       Linux
Inode count:              491520
Block count:              1966080
Reserved block count:     98304
Free blocks:              1910201
Free inodes:              491509
First block:              0
Block size:               4096
Fragment size:            4096
Group descriptor size:    64
Reserved GDT blocks:      959
Blocks per group:         32768
Fragments per group:      32768
Inodes per group:         8192
Inode blocks per group:   512
Flex block group size:    16
Filesystem created:       Fri Sep 27 19:48:23 2024
Last mount time:          Fri Sep 27 19:49:50 2024
Last write time:          Fri Sep 27 19:49:50 2024
Mount count:              1
Maximum mount count:      -1
Last checked:             Fri Sep 27 19:48:23 2024
Check interval:           0 (<none>)
Lifetime writes:          3873 kB
Reserved blocks uid:      0 (user root)
Reserved blocks gid:      0 (group root)
First inode:              11
Inode size:               256
Required extra isize:     32
Desired extra isize:      32
Journal inode:            8
Default directory hash:   half_md4
Directory Hash Seed:      c2ea541c-be50-4fa3-a4e2-563221b47a01
Journal backup:           inode blocks
Checksum type:            crc32c
Checksum:                 0x3512909c
Journal features:         journal_64bit journal_checksum_v3
Journal size:             64M
Journal length:           16384
Journal sequence:         0x00000002
Journal start:            1
Journal checksum type:    crc32c
Journal checksum:         0x1fdaa751

[09/27/24]seed@VM:~$
```

sudo dumpe2fs –h /dev/sdb1

The dumpe2fs command provides detailed information about the ext4 file system, including block size, reserved blocks, and inode structure. It is used to analyze the file system's internal structure and attributes.

has_journal: The filesystem uses a journal to keep track of changes before writing them to the disk. Journaling helps recover from crashes more easily.

ext_attr: This feature supports extended attributes (xattrs), which are name/value pairs that can be associated with files and directories for storing metadata.

resize_inode: Enables the filesystem to be resized dynamically. A special inode is reserved for future resizing.

dir_index: Directories are indexed using a hash tree. This speeds up lookups in large directories.

filetype: This feature stores the file type in the directory entry itself, making directory lookups more efficient.

needs_recovery: Indicates that the filesystem needs recovery via the journal (if a crash happened, the journal will be replayed).

extent: Enables the use of extents (instead of the older block-mapping system). Extents are larger, contiguous blocks of storage and improve performance for large files.

64bit: The filesystem can handle 64-bit block numbers, allowing support for very large filesystems (more than 16 TB).

flex_bg: Flex block groups improve allocation flexibility, meaning files are not tied to specific block groups, increasing performance and disk space usage efficiency.

sparse_super: Reduces the number of superblock backups, which saves space on large filesystems.

large_file: Supports large files larger than 2 GB.

huge_file: Supports files larger than 2 TB.

dir_nlink: Allows directories to contain more than 32,000 subdirectories, increasing the limit from the ext3 filesystem.

extra_isize: The inode size is larger than the standard 128 bytes, allowing extra metadata to be stored in inodes.

metadata_csum: Protects metadata with checksums, ensuring filesystem integrity.

8. **Unmount the USB device:**

```
[09/27/24]seed@VM:~$ sudo umount /dev/sdb1
[09/27/24]seed@VM:~$
```

sudo umount /dev/sdb1

This step safely detaches the USB stick from the mount point, ensuring all data is written to the device before removal. It prevents data corruption by ensuring no files are being accessed during device removal.

9. **Reformat to Windows format :**

To return the USB stick to a Windows-readable format, you can either use:

sudo mkfs.vfat /dev/sdb1

or, use the Windows format tool in File Manager.

The mkfs.vfat command or Windows format tool converts the file system back to FAT32 (VFAT), which is readable by both Windows and Linux. This step is necessary if you want to use the USB stick on a Windows machine again.

Task 2) Memory Layout and the Stack

```
matthew_tran@vm:~$ gedit hello_world-1.c
```

gedit hello_world-1.c

This command opens a file called hello_world in the gedit text editor.

1. **Program 1: hello_world-1.c:**

```c
#include <stdio.h>
int main(void){
printf("hello world\n");
return 0;
}
```

#include <stdio.h>

int main(void){

printf("hello world\n");

return 0;

}

2. **Compile and link the program:**

```
matthew_tran@vm:~$ gcc -W -Wall -c hello_world-1.c
matthew_tran@vm:~$ gcc -o hello_world-1 hello_world-1.o
```

gcc -W -Wall -c hello_world-1.c

gcc -o hello_world-1 hello_world-1.o

The gcc -W -Wall -c and gcc -o commands compile and link the C program hello_world-1.c into an executable. This step transforms your C code into machine code that can be executed by the computer.

3. **Use size to examine the memory layout:**

```
matthew_tran@vm:~$ size hello_world-1 hello_world-1.o
   text    data     bss     dec     hex filename
   1565     600       8    2173     87d hello_world-1
    127       0       0     127      7f hello_world-1.o
```

size hello_world-1 hello_world-1.o

The size command shows the memory segments (text, data, bss) of both the object file and the executable. This is used to see how much memory is occupied by different sections of the program.

4. **Use objdump to inspect object file sections:**

```
matthew_tran@vm:~$ objdump -h hello_world-1.o

hello_world-1.o:     file format elf64-x86-64

Sections:
Idx Name          Size      VMA               LMA               File off  Algn
  0 .text         0000001b  0000000000000000  0000000000000000  00000040  2**0
                  CONTENTS, ALLOC, LOAD, RELOC, READONLY, CODE
  1 .data         00000000  0000000000000000  0000000000000000  0000005b  2**0
                  CONTENTS, ALLOC, LOAD, DATA
  2 .bss          00000000  0000000000000000  0000000000000000  0000005b  2**0
                  ALLOC
  3 .rodata       0000000c  0000000000000000  0000000000000000  0000005b  2**0
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
  4 .comment      0000002b  0000000000000000  0000000000000000  00000067  2**0
                  CONTENTS, READONLY
  5 .note.GNU-stack 00000000  0000000000000000  0000000000000000  00000092  2**0
                  CONTENTS, READONLY
  6 .note.gnu.property 00000020  0000000000000000  0000000000000000  00000098  2**3
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
  7 .eh_frame     00000038  0000000000000000  0000000000000000  000000b8  2**3
                  CONTENTS, ALLOC, LOAD, RELOC, READONLY, DATA
```

objdump -h hello_world-1.o

The objdump -h command provides details about each section (text, data, etc.) within the object file. This helps to understand the memory layout at the assembly level and provides a more in-depth view of memory usage.

5. **Program 2: hello_world-2.c:**

```c
#include <stdio.h>
void first_function(void);
void second_function(int);

int main(void)
{
    printf("hello world\n");
    first_function();
    printf("goodbye goodbye\n");
    return 0;
}

void first_function(void)
{
    int imidate = 3;
    char broiled = 'c';
    void *where_prohibited = NULL;
    second_function(imidate);
    imidate = 10;
}

void second_function(int a)
{
    int b = a;
}
```


#include <stdio.h>

void first_function(void);

void second_function(int);

```
int main(void)
{
   printf("hello world\n");
   first_function();
   printf("goodbye goodbye\n");
   return 0;
}

void first_function(void)
{
   int imidate = 3;
   char broiled = 'c';
   void *where_prohibited = NULL;
   second_function(imidate);
   imidate = 10;
}

void second_function(int a)
{
   int b = a;
}
```

6. **Compile the program:**

```
[09/27/24]seed@VM:~$ gcc -g -o hello_world-2 hello_world-2.c
[09/27/24]seed@VM:~$ ./hello_world2
bash: ./hello_world2: No such file or directory
[09/27/24]seed@VM:~$ ./hello_world-2
hello world
goodbye goodbye
```

gcc -g -o hello_world-2 hello_world-2.c

7. **Start GDB with the program:**

```
[09/27/24]seed@VM:~$ gdb hello_world-2
GNU gdb (Ubuntu 9.2-0ubuntu1~20.04) 9.2
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
/opt/gdbpeda/lib/shellcode.py:24: SyntaxWarning: "is" with a literal. Did you mean "=="?
  if sys.version_info.major is 3:
/opt/gdbpeda/lib/shellcode.py:379: SyntaxWarning: "is" with a literal. Did you mean "=="?
  if pyversion is 3:
Reading symbols from hello_world-2...
gdb-peda$
```

gdb hello_world-2

8. **Set breakpoints:**

```
gdb-peda$ break main
Breakpoint 1 at 0x1149: file hello_world-2.c, line 7.
gdb-peda$ break first_function
Breakpoint 2 at 0x1175: file hello_world-2.c, line 15.
gdb-peda$ break secoand)function
Function "secoand)function" not defined.
gdb-peda$ break second_function
Breakpoint 3 at 0x11a8: file hello_world-2.c, line 24.
gdb-peda$
```

(gdb) break main

(gdb) break first_function

(gdb) break second_function

Using gdb hello_world-2 and setting breakpoints (gdb break main, gdb break first_function, etc.) allows you to control where the program pauses during execution. This is used to step through the program line by line and examine the stack frames at each function call.

9. Runs the program

```
gdb-peda$ run
Starting program: /home/seed/hello_world-2
[----------------------------------registers-----------------------------------]
RAX: 0x555555555149 (<main>:    endbr64)
RBX: 0x5555555551c0 (<__libc_csu_init>: endbr64)
RCX: 0x5555555551c0 (<__libc_csu_init>: endbr64)
RDX: 0x7fffffffe138 --> 0x7fffffffe458 ("SHELL=/bin/bash")
RSI: 0x7fffffffe128 --> 0x7fffffffe43f ("/home/seed/hello_world-2")
RDI: 0x1
RBP: 0x0
RSP: 0x7fffffffe038 --> 0x7ffff7deb0b3 (<__libc_start_main+243>:        mov    edi,eax)
RIP: 0x555555555149 (<main>:    endbr64)
R8 : 0x0
R9 : 0x7ffff7fe0d50 (endbr64)
R10: 0x0
R11: 0x0
R12: 0x555555555060 (<_start>:  endbr64)
R13: 0x7fffffffe120 --> 0x1
R14: 0x0
R15: 0x0
EFLAGS: 0x246 (carry PARITY adjust ZERO sign trap INTERRUPT direction overflow)
[-------------------------------------code-------------------------------------]
   0x555555555139 <__do_global_dtors_aux+57>:   nop    DWORD PTR [rax+0x0]
   0x555555555140 <frame_dummy>:        endbr64
   0x555555555144 <frame_dummy+4>:      jmp    0x5555555550c0 <register_tm_clones>
=> 0x555555555149 <main>:       endbr64
   0x55555555514d <main+4>:     push   rbp
   0x55555555514e <main+5>:     mov    rbp,rsp
   0x555555555151 <main+8>:     lea    rdi,[rip+0xeac]        # 0x555555556004
   0x555555555158 <main+15>:    call   0x555555555050 <puts@plt>
[--------------------------------------stack-------------------------------------]
0000| 0x7fffffffe038 --> 0x7ffff7deb0b3 (<__libc_start_main+243>:       mov    edi,eax)
0008| 0x7fffffffe040 --> 0x7ffff7ffc620 --> 0x5044100000000
0016| 0x7fffffffe048 --> 0x7fffffffe128 --> 0x7fffffffe43f ("/home/seed/hello_world-2")
0024| 0x7fffffffe050 --> 0x100000000
0032| 0x7fffffffe058 --> 0x555555555149 (<main>:        endbr64)
0040| 0x7fffffffe060 --> 0x5555555551c0 (<__libc_csu_init>:     endbr64)
0048| 0x7fffffffe068 --> 0x5675f3e50db34c52
0056| 0x7fffffffe070 --> 0x555555555060 (<_start>:      endbr64)
[------------------------------------------------------------------------------]
Legend: code, data, rodata, value

Breakpoint 1, main () at hello_world-2.c:7
7       {
gdb-peda$
```

(gdb) run

10. View the call stack:

```
gdb-peda$ bt
#0  main () at hello_world-2.c:7
#1  0x00007ffff7deb0b3 in __libc_start_main (main=0x555555555149 <main>, argc=0x1,
    argv=0x7fffffffe128, init=<optimized out>, fini=<optimized out>,
    rtld_fini=<optimized out>, stack_end=0x7fffffffe118) at ../csu/libc-start.c:308
#2  0x000055555555508e in _start ()
```

(gdb) bt

Running the program in GDB with gdb run stops at the first breakpoint, and the bt (backtrace) command lets you view the sequence of function calls (stack frames). This is important for understanding how function calls are stacked in memory and how variables are handled.

11. code line by line:

```
gdb-peda$ step
[----------------------------------registers-----------------------------------]
RAX: 0x555555555149 (<main>:    endbr64)
RBX: 0x5555555551c0 (<__libc_csu_init>: endbr64)
RCX: 0x5555555551c0 (<__libc_csu_init>: endbr64)
RDX: 0x7fffffffe138 --> 0x7fffffffe458 ("SHELL=/bin/bash")
RSI: 0x7fffffffe128 --> 0x7fffffffe43f ("/home/seed/hello_world-2")
RDI: 0x1
RBP: 0x7fffffffe030 --> 0x0
RSP: 0x7fffffffe030 --> 0x0
RIP: 0x555555555151 (<main+8>:  lea    rdi,[rip+0xeac]        # 0x555555556004)
R8 : 0x0
R9 : 0x7ffff7fe0d50 (endbr64)
R10: 0x0
R11: 0x0
R12: 0x555555555060 (<_start>:  endbr64)
R13: 0x7fffffffe120 --> 0x1
R14: 0x0
R15: 0x0
EFLAGS: 0x246 (carry PARITY adjust ZERO sign trap INTERRUPT direction overflow)
[-------------------------------------code-------------------------------------]
   0x555555555149 <main>:       endbr64
   0x55555555514d <main+4>:     push   rbp
   0x55555555514e <main+5>:     mov    rbp,rsp
=> 0x555555555151 <main+8>:     lea    rdi,[rip+0xeac]        # 0x555555556004
   0x555555555158 <main+15>:    call   0x555555555050 <puts@plt>
   0x55555555515d <main+20>:    call   0x555555555175 <first_function>
   0x555555555162 <main+25>:    lea    rdi,[rip+0xea7]        # 0x555555556010
   0x555555555169 <main+32>:    call   0x555555555050 <puts@plt>
[-------------------------------------stack-------------------------------------]
0000| 0x7fffffffe030 --> 0x0
0008| 0x7fffffffe038 --> 0x7ffff7deb0b3 (<__libc_start_main+243>:       mov    edi,eax)
0016| 0x7fffffffe040 --> 0x7ffff7ffc620 --> 0x5044100000000
0024| 0x7fffffffe048 --> 0x7fffffffe128 --> 0x7fffffffe43f ("/home/seed/hello_world-2")
0032| 0x7fffffffe050 --> 0x100000000
0040| 0x7fffffffe058 --> 0x555555555149 (<main>:        endbr64)
0048| 0x7fffffffe060 --> 0x5555555551c0 (<__libc_csu_init>:     endbr64)
0056| 0x7fffffffe068 --> 0xdf4e25e298c222b6
[------------------------------------------------------------------------------]
Legend: code, data, rodata, value
8           printf("hello world\n");
gdb-peda$
```

(gdb) step

12. Print the value of imidate in first_function:

```
gdb-peda$ print imidate
$2 = 0x5555
```

(gdb) print imidate

The step command allows you to execute the program one line at a time, while print imidate shows the value of the local variable in the current function. This helps in visualizing how variables and function calls interact within the stack.

13. Quit GDB:

```
gdb-peda$ quit
[09/27/24]seed@VM:~$
```

(gdb) quit

After you have finished stepping through the code and examining the memory, use the quit command to exit GDB. This ends the debugging session and returns you to the command line.